The Maurice Sievan Archive: Data Model & Front-End Resolution

How a record travels from Supabase to a static HTML page

Repository	`lalaf02/sievan-web`
Read at commit	`5efb4c3506a1aeeebc7cb381eccfc069f169f115` (branch `claude/brave-galileo-vn8x46`)
Prepared	18 October 2026
Scope	Written from the repository's migrations, scripts, source and documentation. The live database was not queried. Row counts are the ones the repo's documentation records. Every file this document relies on is listed in Appendix A.

Contents

1. The big picture
2. Layer 1: The tables in Supabase
3. Layer 2: The `api.v_*` views ("the seam")
4. Layer 3: The build scripts
5. Layer 4: The front end
6. Rules the model protects
7. Appendix A: Sources cited
8. Appendix B: All Supabase migrations, in order
9. Appendix C: Notes and limits

---

1. The big picture

The published site **never talks to Supabase while someone is browsing**. It has no API routes, no login and no runtime data fetching. Supabase is the master copy. It is read once, when the site is built, and the result is frozen into a JSON file committed to git. Next.js then turns that file into about 265 plain HTML pages (`output: 'export'`), designed to still open from a USB stick in twenty years.

```
Supabase project "Sievan"  (12 tables + Storage)   ← THE SOURCE OF RECORD; the curator edits here
        │  api.v_* views reshape the tables into the shape the site expects  ← "THE SEAM"
        ▼
scripts/fetch-data.mjs  +  scripts/media.mjs        ← 16 small view reads, ~300 rows; syncs Storage → public/
        ▼
scripts/build-data.mjs                              ← schema check (ajv), transcript parsing,
        │                                                23 derived indexes, writes the bundle
        ▼
data/archive.generated.json  +  data/transcripts/*.json   ← COMMITTED to git
        │  plain `import` at build time. Never fetched.
        ▼
lib/data.ts  →  Server Components  →  static HTML in out/
```

Adapted from docs/data-pipeline.md, "Backend architecture"

The archive's generated data (`data/`) and scans (`public/scans/`) are committed on purpose. They are a durable copy of the archive in version control, and every change to the record shows up as a reviewable diff. They are **not a fallback**: `build-data.mjs` requires Supabase credentials and fails without them.

---

2. Layer 1: The tables in Supabase

Defined in `supabase/migrations/20260830165100_create_seven_table_core.sql`. The database was consolidated from 21 tables to 12 on 30 August 2026.

Content tables

Table	Rows	What it holds
`artworks`	25	Physical works. `artwork_type` splits it into `'drawing'` (the 25 sheets the estate holds) and `'painting'` (0 today). A drawing must have an `archive_id` (accession number).
`articles`	111	Every document: 51 containers (bundles, catalogues, posters…) and the 60 press clippings inside them. A clipping points to its container through `parent_article_id`, and a check constraint enforces `(article_type = 'press_notice') = (parent_article_id is not null)`.
`artwork_mentions`	57	"Attested works": paintings a source *names* but the archive doesn't hold. Each row stores the verbatim `quote` it rests on and has exactly one source column set: `source_artwork_id`, `source_article_id` or `source_interview_id`. Ids follow `MS-AW-#####`.
`interviews`	7	Oral-history videos, with the full transcript text stored in `transcript_text`.
`events`	15	Exhibitions (plus award, historical-context and other event types, currently none). `name` is nullable because some shows are known only by venue and year.
`media_assets`	281	A list of every file the archive holds. **The build doesn't read it**; images come from a Storage listing (`api.v_media_manifest`). Ids are UUID v5 values derived from `(storage_bucket, storage_path)`.

Lookup tables and login

Table	Rows	Role
`people`	29	Critics, gallery owners, interview subjects and others. `roles` can hold several values; `aliases` feed name matching.
`publications`	30	Newspapers and magazines; `aliases` fold variant mastheads onto one record.
`places`	25	A list of places the evidence names. No coordinates, by design.
`collections`	3	The physical collections (boxes) objects belong to.
`artwork_mention_places`	37	Links a mention to a place with a `role` (`depicted`, `made_at`, `shown_at` or `held_at`), because a place a painting shows is not a place it was exhibited. `certain` is null on 33 of 37 rows, meaning the source is silent, which is not the same as false.

`profiles`	1	Curator login. Every access rule calls `private.is_curator()`, which reads this table. Anonymous users are granted nothing.

Three design ideas

1. **Separate columns for "as written" and "cleaned up".** For example `title_as_written` sits beside `title`, and `dimensions_as_written` / `price_as_written` sit beside parsed values. The source's exact wording is never overwritten.
2. **Nullable where the source is silent.** Making a field required would force someone to invent a value, which the archive forbids ("Never fabricate a record").
3. **Columns that split a table keep the evidence apart.** Drawings and paintings share `artworks`, but they always leave through different views, so their counts are never added together.

Excerpt: the polymorphic source and the one-way link to the catalogue on `artwork_mentions`:

```
  source_artwork_id   text references public.artworks(id),
  source_article_id   text references public.articles(id),
  source_interview_id text references public.interviews(id),
  ...
  quote text not null check (length(quote) >= 4),
  ...
  artwork_id            text references public.artworks(id),
  identification_basis  text check (identification_basis in (
                          'title_and_dimensions', 'artist_number', 'photograph',
'curator_judgement')),
  identification_status text not null default 'unresolved'
  ...
  check (num_nonnulls(source_artwork_id, source_article_id, source_interview_id) = 1)
```

`supabase/migrations/20260830165100_create_seven_table_core.sql (excerpted; "..." marks omitted lines)`

---

3. Layer 2: The `api.v_*` views ("the seam")

The build reads **views, never the tables directly**. Each view outputs exactly the shape the site's JSON Schema (`schema/data_model.schema.json`) expects. That's why the 21-to-12-table consolidation changed **no line** of `app/`, `lib/` or `components/`: the views absorbed the reshape, and `scripts/check-parity.mjs` proved the generated bundle byte-identical.

View	Bundle key	What it does
`v_archive_objects`	`archiveObjects` (76)	`UNION ALL` of `artworks` where type = `'drawing'` and the non-clipping rows of `articles`. Drawings get an `artwork` field; documents get `null`. Builds `scan_files` from `media_assets` and `article_ids` from child clippings.
`v_news_articles`	`newsArticles` (60)	`articles` where type = `'press_notice'`; renames `parent_article_id` to `archive_object_id`, `title` to `headline`, `event_id` to `exhibition_id`.

`v_attested_works`	`attestedWorks` (57)	Turns the three source columns back into `source_type` / `source_id`; gathers the place links into `place_refs`; exposes `artwork_id` as `painting_id`.
`v_paintings`	`paintings` (0)	`artworks` where type = `'painting'`.
`v_persons`, `v_exhibitions`, `v_publications`, `v_places`, `v_collections`, `v_video_assets`, `v_transcript_texts`	matching keys	Rename and reshape the lookup and interview tables.
`v_commentary`, `v_commentary_relations`, `v_painting_exhibitions`, `v_painting_historical_context`, `v_historical_events`, `v_scholarship`	matching keys (0 each)	Typed empty views with no table behind them. They exist so code waiting on future rows (for example `components/Relations.tsx`) still resolves.

The drawing half of `v_archive_objects`, where the `artwork` field is created:

```
            -- Presence is the semantic signal: null means "a document about the art",
            -- an object means "a work of art the archive holds".
            jsonb_build_object(
              'medium_stated', w.medium,
              'support',       w.support,
              'signed',        w.signed_text,
              'sheet_count',   w.sheet_count) as artwork,
            w.sort_order
    from public.artworks w
    where w.artwork_type = 'drawing'
    union all
```

supabase/migrations/20260830165525_hide_sort_order_from_archive_objects_view.sql

The paintings view, which partitions the same table:

```
create view api.v_paintings as
  select id, title, date_display as date_text, year_start as date_earliest,
         year_end as date_latest, medium, dimensions_text as dimensions,
         current_location, image_ref, catalogue_status as catalog_status, notes
  from public.artworks where artwork_type = 'painting'
  order by sort_order, id;
```

supabase/migrations/20260830165455_reshape_seed_views_onto_seven_table_core.sql

Two rules the views must follow

- **Never `select *`.** The schema sets `additionalProperties: false`, so one stray `created_at` fails every row. That's why `v_archive_objects` wraps its `UNION` in a subquery: it orders by `sort_order` without putting that column in the output.
- **Always `coalesce(jsonb_agg(...), '[]')`.** Building a list from zero rows returns `NULL`, and lists like `scan_files` are required. `jsonb_strip_nulls` is used only on `place_refs`, so a missing `certain` stays missing

instead of becoming false.

---

4. Layer 3: The build scripts

`scripts/fetch-data.mjs`

Holds a `VIEWS` map from each view to a bundle key and fetches all of them in parallel through `scripts/supabase.mjs`. Row order comes from each view's `ORDER BY sort_order`, so the curator's sequence survives.

```
export const VIEWS = {
  v_collections:                'collections',
  v_publications:               'publications',
  v_persons:                    'persons',
  v_exhibitions:                'exhibitions',
  v_video_assets:               'videoAssets',
  v_paintings:                  'paintings',
  // Places load before attestations, which reference them.
  v_places:                     'places',
  v_attested_works:             'attestedWorks',
  ...
  v_archive_objects:            'archiveObjects',
  v_news_articles:              'newsArticles',
};
```

scripts/fetch-data.mjs (excerpted)

`scripts/build-data.mjs`

1. **Validate:** checks every row against `schema/data_model.schema.json` with ajv (draft 2020-12), using a `DEFS` map from bundle key to schema type (e.g. `archiveObjects: 'ArchiveObject'`). Fails with the offending record's id.
2. **Enforce archive rules:** e.g. `artwork` only on `work_on_paper` rows, and `artwork.signed` must appear verbatim in the row's description.
3. **Parse transcripts** into pages and paragraphs (`data/transcripts/*.json`).
4. **Derive 23 indexes** into `derived`: `facets`, `articlesByObject`, `articlesByPublication`, `articlesByAuthor`, `exhibitionsByObject`, `articlesByExhibition`, `exhibitionsByArticle`, `personMentions`, `personTranscriptMentions`, `attestationsByObject`, `attestationsByPlace`, `exhibitionsByPlace`, `placeChildren`, `placeUsage`, `attestationsByTitleKey`, `undatedAttestations`, `publicationMergeGroups`, `timeline`, `undatedVideos`, `objectsWithImagery`, `coverByObject`, `clips`, `clipsByVideo`, and `counts`.
5. **Sync media** from Storage into `public/` (via `scripts/media.mjs`), skipping files whose size already matches.
6. **Write** `data/archive.generated.json`, and create or delete the generated route `app/works/[paintingId]/`. A static export refuses a page template that would produce zero pages, and there are zero paintings.

Four derived indexes are guesses, and the site labels them that way. `personMentions` and `personTranscriptMentions` are name matches, so the site says "mentioned", never "authored". The transcript version uses full names only, never surnames. `articlesByExhibition` / `exhibitionsByArticle` link a clipping to a show only when the year matches *and* a distinctive venue word appears in the clipping. `attestationsByTitleKey` groups similar titles as an open question, never a merge.

Checks (gates)

Script	Catches
`check-data.mjs`	Broken links between records; scans, media or transcripts referenced but missing from disk; a place nothing points at; a `parent_id` cycle.
`check-quotes.mjs`	A curated quote whose anchor phrase is no longer on its transcript page, and an attested work whose verbatim quote isn't in its source's own text.
`check-export.mjs`	Dead internal links and a page count that collapsed (runs after `next build`).
`check-parity.mjs`	Any difference in the bundle against a saved baseline. Run on every schema change.

`npm run data` runs `build-data` → `check-data` → `check-quotes`; `npm run build` runs `next build` then `check-export`.

---

5. Layer 4: The front end

`lib/data.ts`: the only file that reads the bundle

It imports the JSON directly (no network request), checks its shape at load, builds lookup maps by id, and provides every accessor the pages use. Types live in `lib/types.ts`.

```
import bundle from '@/data/archive.generated.json';
...
validateBundle(bundle);
export const archive = bundle as Archive;
export const counts = archive.derived.counts;

const index = <T extends { id: string }>(rows: T[]): Map<string, T> =>
  new Map(rows.map((r) => [r.id, r]));

const objects = index(archive.archiveObjects);
...
export const getObject = (id: string): ArchiveObject | undefined => objects.get(id);
...
export const catalogueWorksOnPaper = (): ArchiveObject[] =>
  allObjects.filter((o) => o.artwork && hasImagery(o.id)).sort(byDateUndatedLast);
```

lib/data.ts (excerpted)

Other helpers include `articlesForObject`, `attestationsForObject`, `exhibitionsForObject`, `getPerson`, `getPublication`, `mentionsForPerson`, `loadTranscript`, `coverFor` and `hasImagery`. Most are thin lookups into the precomputed `derived` indexes.

A page that uses it: `app/archive/objects/[objectId]/page.tsx`

Pages are **server components** that run at build time. `generateStaticParams` turns each bundle row into one HTML page:

```
export function generateStaticParams() {
  return allObjects.map((o) => ({ objectId: o.id }));
}
...
export default async function ObjectPage({ params }: Props) {
  const { objectId } = await params;
  const object = getObject(objectId);
  if (!object) notFound();

  const articles = articlesForObject(object.id);
  const attestations = attestationsForObject(object.id);
  const exhibitions = exhibitionsForObject(object.id);
  ...
      <RecordHeader
        backHref={object.artwork ? '/works/' : '/archive/'}
```

`app/archive/objects/[objectId]/page.tsx (excerpted). Rendered with components/Record.tsx.`

One record, traced end to end

1. A drawing is a row in `public.artworks` with `artwork_type = 'drawing'`.
2. `api.v_archive_objects` emits it with `object_type = 'work_on_paper'` and an `artwork` field built from `medium`, `support`, `signed_text` and `sheet_count`. It also attaches `scan_files` from `media_assets`.
3. `fetch-data.mjs` stores it under `archiveObjects`; `build-data.mjs` validates it as an `ArchiveObject`, adds it to indexes like `coverByObject`, and counts it in `worksOnPaperCatalogued`.
4. `lib/data.ts` serves it through `getObject()` and `catalogueWorksOnPaper()`.
5. The object page renders it as a catalogue entry linking back to **/works/** (the Catalogue Raisonné) instead of /archive/.

Client components

Only 10 of 28 components run in the browser (filter rails, search, the header and similar). They receive data as props and keep filter state in the URL through `lib/useUrlState.ts`, never `useSearchParams`. Its server snapshot is deliberately empty, so the unfiltered list is what ends up in the HTML and **everything still reads with JavaScript turned off**.

---

6. Rules the model protects

- `counts.paintings` **(0) is never added to** `counts.worksOnPaperCatalogued` **(25)**, and no single total is shown across the four grades of evidence (catalogue plates, gallery reproductions, sheets the estate holds, and paintings only named on sheets).
- **Attested works are evidence, not catalogue entries.** The 57 have their own `MS-AW-#####` ids and are deep-linked as `/works/attested/#MS-AW-…`. Their counts are never added to anything else.
- **Never fabricate a record.** Gaps are shown with `Pending` and `Absent`; nullable columns exist so nothing has to be invented.
- **Never present a guess as a fact.** Inferred links are labelled and show their working.
- **No box or folder identifier in user-facing text.** `collection_id`, `folder_no` and `seq` are internal shelving only.

- **Never hand-edit `data/archive.generated.json`.** Change the data in Supabase, then run `npm run data`.
- **Schema change procedure:** copy the bundle to a baseline, migrate, run `npm run data`, then `node scripts/check-parity.mjs <baseline>`. It must report no difference.
- **Backup path:** `export-seeds.mjs` dumps the database to seed-shaped JSON and `seed-supabase.mjs` loads it back. They are inverses, tested by a round-trip `diff`, so the archive can be moved off this Supabase account.

Appendix A: Sources cited

Every repository file this document draws on. Links point to the exact commit read (`5efb4c3`), so they keep matching this document even if the files change later.

Path	What it is	Used in
Project documentation		
`CLAUDE.md`	Project overview and the non-negotiable rules	§1, §6
`AGENTS.md`	Note that Next 16 differs from older versions	context
`docs/architecture.md`	File layout, corpus size, key utilities table	§1, §5
`docs/data-pipeline.md`	Pipeline diagram, entity model and row counts, view rules, derived indexes, gates	§1–§4, §6
`docs/frontend.md`	No-JavaScript rule, static export limits, dynamic routes	§4, §5
`docs/editorial.md`	Editorial rules (no fabrication, label guesses, no box ids)	§6 (cited via CLAUDE.md)
`docs/verification.md`	Ways the repo can report a pass while broken (e.g. trap 1)	§1 (cited via data-pipeline.md)
Database (Supabase migrations)		
`…/20260830165100_create_seven_table_core.sql`	Current core tables, constraints, security	§2
`…/20260830165455_reshape_seed_views_onto_seven_table_core.sql`	The `api.v_*` views	§3
`…/20260830165525_hide_sort_order_from_archive_objects_view.sql`	Current `v_archive_objects`	§3
All 27 migrations are listed in Appendix B.		
Schema		
`schema/data_model.schema.json`	JSON Schema every row is checked against	§3, §4
Build scripts		
`scripts/fetch-data.mjs`	Reads the views; `VIEWS` map	§4
`scripts/supabase.mjs`	Supabase client helper (`selectAll`)	§4

`scripts/build-data.mjs`	Validation, derived indexes, bundle output	§4
`scripts/media.mjs`	Storage sync; `uuidFor`	§2, §4
`scripts/upload-media.mjs`	Uploads media and registers `media_assets` rows	§2
`scripts/check-data.mjs`	Gate: integrity	§4
`scripts/check-quotes.mjs`	Gate: quotes	§4
`scripts/check-export.mjs`	Gate: links and page count	§4
`scripts/check-parity.mjs`	Gate: schema-change parity	§3, §4, §6
`scripts/export-seeds.mjs`	Dumps the database to seed JSON	§6
`scripts/seed-supabase.mjs`	Loads seed JSON into the database	§6
Front end and data		
`lib/data.ts`	The only reader of the bundle; all accessors	§5
`lib/types.ts`	TypeScript types for bundle entities	§5
`lib/useUrlState.ts`	URL-held filter state that works without JavaScript	§5
`app/archive/objects/[objectId]/page.tsx`	Worked example of a record page	§5
`components/Record.tsx`	Record-page building blocks (`RecordHeader`, `Fact`, `Absent` …)	§5
`data/archive.generated.json`	The generated bundle (output; never hand-edited)	§1, §4, §5

Appendix B: All Supabase migrations, in order

In `supabase/migrations/`. The **bold** ones define the current model; the earlier ones record the path to it (the 21-table entity model and its replacement).

Migration file
`20260829222659_sievan_archive_core_schema.sql`
`20260829222800_sievan_archive_security_search_storage.sql`
`20260829222808_sievan_archive_staging_deny_policies.sql`
`20260829222830_sievan_archive_performance_indexes.sql`

`20260829223140_connect_existing_storage_assets.sql`
`20260829223157_resolve_storage_links_on_archive_import.sql`
`20260829223831_create_simplified_archive_model.sql`
`20260829223923_remove_legacy_archive_model_v2.sql`
`20260829223931_remove_last_legacy_table.sql`
`20260829223957_remove_duplicate_media_index.sql`
`20260830003454_bootstrap_admin_and_storage_access.sql`
`20260830112815_open_admin_anon_access.sql`
`20260830115611_sievan_archive_entity_model.sql`
`20260830115857_sievan_archive_seed_shaped_views.sql`
`20260830115922_expose_api_schema_to_postgrest.sql`
`20260830120042_preserve_curator_row_order.sql`
`20260830160500_auth_email_allowlist.sql`
`20260830161439_drop_touch_trigger_on_child_tables.sql`
`20260830163650_pin_touch_updated_at_search_path.sql`
`20260830163748_expose_migration_history_for_repo_sync.sql`
`20260830165100_create_seven_table_core.sql`
`20260830165322_migrate_rows_into_seven_table_core.sql`
`20260830165455_reshape_seed_views_onto_seven_table_core.sql`
`20260830165525_hide_sort_order_from_archive_objects_view.sql`
`20260830165827_drop_legacy_entity_model.sql`
`20260830170101_index_artwork_mention_counterparty.sql`
`20260830170449_derive_media_asset_ids_from_their_row.sql`

Appendix C: Notes and limits

- This document was written from the repository at the commit above. **The live Supabase database was not queried**; if a curator has edited it since, current row counts may differ from those shown.
- Row counts come from `docs/data-pipeline.md` and `docs/architecture.md`, not from a fresh `npm run data`.
- Code excerpts are copied verbatim from the files cited beneath them; "..." marks omitted lines.
- Section 2's "Three design ideas" and the "One record, traced end to end" walkthrough are this document's own summaries of the cited files, not quotations.